

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 2

Mock Interview

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO5: Students will be able to evaluate advanced machine learning concepts such as reinforcement learning, Monte Carlo methods, sampling techniques, policy iteration, value iteration, and temporal difference learning. They will understand computational learning theory concepts including VC dimension, sample complexity, and mistake-bound analysis. Students will be capable of selecting and optimizing advanced learning strategies for intelligent systems and complex applications. (BL-6)

Assessment Tool Weightage: 20%

QUESTIONS:

Q1. Autoencoder Design

- What is an autoencoder and what are its main components in a neural network architecture?
- How would you design an autoencoder for reducing the dimensionality of a high-dimensional dataset?
- What factors would you consider when deciding the size of the encoder and decoder layers?
- Why is the bottleneck layer important in an autoencoder design?
- How would you choose an appropriate activation function for the encoder and decoder of an autoencoder?

Q2. Stochastic Encoders and Decoders

- What is a stochastic encoder, and how does it differ from a deterministic encoder?
- Why is randomness introduced into the encoding process of a stochastic autoencoder?
- How does a stochastic decoder reconstruct data from a probabilistic latent representation?
- What are the advantages of using stochastic encoders and decoders in generative models?
- How does a stochastic encoder represent uncertainty in the input data?

Q3. Deep Belief Network vs Deep Boltzmann Machine

- What is the main architectural difference between a Deep Belief Network (DBN) and a Deep Boltzmann Machine (DBM)?
- How are connections between hidden layers organized differently in a DBN and a DBM?
- Why is a DBN considered a hybrid model containing both directed and undirected connections?
- How does a DBM differ from a DBN in terms of the direction of connections between layers?
- How does the training procedure of a DBN differ from that of a DBM?

Q4. Generative Adversarial Networks

- What is a Generative Adversarial Network, and what are its two main components?
- How does the generator in a GAN create new data samples?

- What is the primary role of the discriminator in a GAN?
- How do the generator and discriminator compete with each other during GAN training?
- Why is balancing the generator and discriminator important for stable GAN training?

Q5. Integrated Deep Learning Solution

- What do you understand by an integrated deep learning solution, and what are its major components?
- How would you combine different deep learning models to solve a complex real-world problem?
- How would you decide whether to use an autoencoder, DBN, DBM, or GAN in an integrated solution?
- What factors should be considered when designing an end-to-end deep learning pipeline?
- How can feature extraction and representation learning be integrated into a deep learning solution?

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism

MOCK INTERVIEW — ANSWERS

Q1. Autoencoder Design

What is an autoencoder and what are its main components?

An autoencoder is an unsupervised neural network that learns to reconstruct its own input by first compressing it into a lower-dimensional code and then rebuilding the original data from that code. Its main components are the Encoder, which maps the input x to a latent representation z ; the Latent/Bottleneck layer, which holds the compressed code; the Decoder, which reconstructs the input as $\hat{x}$ from z ; and the Reconstruction Loss, which measures the difference between x and $\hat{x}$ (usually Mean Squared Error) and drives training.

How would you design one for dimensionality reduction?

I would use an undercomplete autoencoder where the bottleneck size k is much smaller than the input size n . For example, for a 500-dimensional dataset I might use an encoder of $500 \rightarrow 256 \rightarrow 64 \rightarrow 16$ and a mirrored decoder of $16 \rightarrow 64 \rightarrow 256 \rightarrow 500$. I would train it end-to-end using MSE reconstruction loss with the Adam optimizer, then use only the trained encoder to project new data into the 16-dimensional latent space for downstream tasks such as visualization or classification.

What factors decide the size of encoder and decoder layers?

- The intrinsic dimensionality/complexity of the data — more complex data needs a larger bottleneck.
- The amount of available training data — deeper/wider networks need more data to avoid overfitting.
- The downstream task requirements — e.g., how much information the latent code must preserve.
- Computational and latency constraints of the deployment environment.
- Empirical validation — trying a few bottleneck sizes and picking the one with the best reconstruction-vs-compression trade-off on a validation set.

Why is the bottleneck layer important?

The bottleneck forces the network to discard redundant and noisy information and retain only the most informative features needed to reconstruct the input. Without this compression constraint, the network could simply learn an identity mapping and would learn nothing useful about the structure of the data.

How would you choose the activation function?

For hidden layers in the encoder and decoder, I would generally use ReLU because it avoids vanishing gradients and trains efficiently. For the final output layer of the decoder, I would match the activation to the data range — Sigmoid if inputs are normalized to $[0, 1]$, Tanh if

inputs are normalized to $[-1, 1]$, or a Linear activation if the data is unbounded/continuous, so that the reconstructed output can actually reach the same range as the original input.

Q2. Stochastic Encoders and Decoders

What is a stochastic encoder and how does it differ from a deterministic one?

A deterministic encoder maps a given input x to exactly one fixed latent vector $z = f(x)$. A stochastic encoder instead outputs the parameters of a probability distribution over the latent space — typically a mean $\mu(x)$ and variance $\sigma^2(x)$ of a Gaussian — and the actual latent code z is sampled from this distribution, so the same input can map to slightly different codes on different passes.

Why is randomness introduced into the encoding process?

Randomness is introduced so the model learns a smooth, continuous latent space rather than memorizing single points for each input. This makes the representation more robust, prevents the network from overfitting to exact training examples, and — most importantly — enables the latent space to be sampled from later to generate new, realistic data, which a purely deterministic encoder cannot support.

How does a stochastic decoder reconstruct data from a probabilistic latent code?

The stochastic decoder takes a sampled latent vector z (drawn from the encoder's distribution, or from the prior during generation) and outputs the parameters of a distribution over the data space, for example the mean of a Gaussian or Bernoulli distribution over pixel values; the final reconstruction is obtained either by taking this mean directly or by sampling from that output distribution.

Advantages of stochastic encoders/decoders in generative models

- They enable proper sampling of new, never-seen data points from the learned latent distribution.
- They produce a smoother, more continuous, and more interpolatable latent space.
- They naturally model uncertainty in the data rather than forcing a single fixed encoding.
- They reduce overfitting compared to deterministic point-estimate encodings.

How does a stochastic encoder represent uncertainty?

The variance term $\sigma^2(x)$ that the encoder outputs directly represents uncertainty: a small variance means the model is confident about where this input should sit in latent space, while a large variance means the model is less certain, allowing the latent code to spread over a wider region — this is exactly how models such as Variational Autoencoders quantify and

propagate uncertainty.

Q3. Deep Belief Network vs Deep Boltzmann Machine

What is the main architectural difference between a DBN and a DBM?

A Deep Belief Network is a hybrid, mostly directed model: only the top two layers form an undirected Restricted Boltzmann Machine, while all the lower layers are connected by directed, top-down generative connections. A Deep Boltzmann Machine, in contrast, is fully undirected — every pair of adjacent layers is connected symmetrically, so the entire network is one large Boltzmann Machine rather than a stack with directed connections at the bottom.

How are hidden-layer connections organized differently?

In a DBN, each hidden layer receives influence in a top-down directed manner from the layer above (except for the top RBM, which is undirected in both directions). In a DBM, every hidden layer is connected symmetrically/bidirectionally to both the layer below and the layer above, so information can flow both upward and downward through every layer simultaneously during inference.

Why is a DBN considered a hybrid model?

A DBN is called hybrid because its topmost two layers form an undirected RBM (providing an associative-memory-style generative prior), while the connections between all the remaining lower layers are directed, belief-network-style connections that generate data top-down once the top RBM has been sampled — so it mixes both undirected and directed connection types in one model.

How does the direction of connections differ between DBM and DBN?

In a DBN, only the top pair of layers is undirected; the rest of the network has connections that point downward from hidden to visible layers, like a Bayesian belief network. In a DBM, there is no directionality anywhere — every layer-to-layer connection is undirected and symmetric, allowing the model to combine bottom-up and top-down evidence jointly during inference, which gives DBMs a more robust way of handling ambiguous or noisy inputs.

How does training differ between a DBN and a DBM?

A DBN is trained with fast greedy layer-wise pretraining, where each RBM is trained one at a time using Contrastive Divergence, treating the previous layer's hidden activations as the next layer's visible data; this is efficient but only approximates the joint distribution. A DBM must be trained with a joint, more expensive procedure — mean-field variational inference is used to approximate the posterior over all hidden layers together, followed by a persistent-Contrastive-Divergence-based update of all layers simultaneously, because every layer influences every other layer bidirectionally.

Q4. Generative Adversarial Networks

What is a GAN and what are its two main components?

A Generative Adversarial Network is a generative model made of two neural networks trained together in a competitive setting: the Generator, which tries to produce fake data that looks real, and the Discriminator, which tries to correctly distinguish real data from the Generator's fake data. The two networks are trained simultaneously so that the Generator keeps improving to fool an increasingly accurate Discriminator.

How does the generator create new data samples?

The generator takes a random noise vector z , typically sampled from a standard normal distribution, and passes it through a series of (transposed convolution or fully connected) layers that progressively transform this noise into a structured output, such as an image, that resembles the training data distribution.

What is the primary role of the discriminator?

The discriminator acts as a binary classifier that receives either a real training sample or a generator-produced sample and outputs the probability that the given input is real; its role is to provide a learning signal that tells the generator how far its output still is from looking realistic.

How do the generator and discriminator compete during training?

Training alternates between the two: first the discriminator is updated to get better at telling real images from the generator's fakes, then the generator is updated to produce images that are more likely to fool the current discriminator. This is a min-max game — the discriminator tries to maximize its classification accuracy while the generator tries to minimize it — and over many iterations both networks improve together until the generator's outputs become nearly indistinguishable from real data.

Why is balancing the generator and discriminator important?

If the discriminator becomes too strong too quickly, it can perfectly separate real from fake samples, giving the generator a vanishing gradient with no useful signal to improve from. Conversely, if the generator becomes too strong or the discriminator too weak, the discriminator gives poor feedback and the generator may collapse to producing a limited variety of outputs (mode collapse). Careful balancing — through learning-rate tuning, update ratios, or techniques like label smoothing — keeps both networks learning together for stable convergence.

Q5. Integrated Deep Learning Solution

What is an integrated deep learning solution and its major components?

An integrated deep learning solution combines multiple specialized deep learning models into a single pipeline so that the strengths of each model address a different part of a complex, real-world problem. Its major components typically include a data pre-processing/feature extraction stage, one or more representation-learning or generative models (autoencoders, DBNs, DBMs, or GANs), a task-specific model built on top of the learned representation (classifier, recommender, etc.), and an evaluation/feedback loop that monitors and improves the overall pipeline.

How would you combine different models to solve a complex problem?

I would typically use an autoencoder or DBN first as an unsupervised feature extractor to reduce raw, high-dimensional or noisy data into a compact, meaningful representation. This representation can then feed a supervised model for the main task (e.g., classification or forecasting), while a GAN can be added separately or in parallel to augment limited training data or generate additional realistic synthetic samples that improve the robustness of the overall pipeline.

How would you decide whether to use an autoencoder, DBN, DBM, or GAN?

- Autoencoder — when the goal is dimensionality reduction, denoising, or compact feature learning from labeled or unlabeled data with a clear reconstruction objective.
- DBN — when a fast, greedy way to learn a hierarchical generative model from largely unlabeled data is needed, often as a pretraining step before fine-tuning.
- DBM — when bidirectional, joint reasoning between top-down and bottom-up evidence is important and enough computation/data is available for its more expensive training.
- GAN — when the primary need is generating new, realistic, diverse synthetic samples, such as for data augmentation or content generation.

What factors should be considered when designing an end-to-end pipeline?

- Data availability, quality, and whether labels exist for the target task.
- Computational budget and latency requirements for training and inference.
- How well each model's strengths align with the specific sub-problem it is assigned to solve.
- Interpretability and evaluation requirements for each stage of the pipeline.
- Scalability and maintainability of the combined system in production.

How can feature extraction and representation learning be integrated?

The encoder portion of an autoencoder, or the hidden layers of a trained DBN/DBM, can be reused as a fixed or fine-tunable feature-extraction front end that converts raw input into a

compact latent representation. This representation is then passed directly into the next stage of the pipeline — for example a classifier, a recommender system, or even a GAN's generator/discriminator — so that representation learning becomes a shared, reusable building block across the entire integrated solution rather than a separate, standalone step.